Waypoint-to-Waypoint Planning and Navigation

Waypoint-to-Waypoint Planning and Navigation

1. Course Content
2. Preparation
3. Waypoint-to-Waypoint Navigation
   - 3.1 Navigation When the Starting Position Is Not on the Road Network
   - 3.2 Requesting Road Network Navigation Through the Action Service Interface
4. Source Code Analysis
5. Parameter Debugging
   - 5.1 Navigation Controller Debugging
   - 5.2 Obstacle Avoidance and Costmap Debugging

1. Course Content

Basic

- Start the road network navigation feature and master waypoint-to-waypoint navigation using the roadnet_nav visualization panel

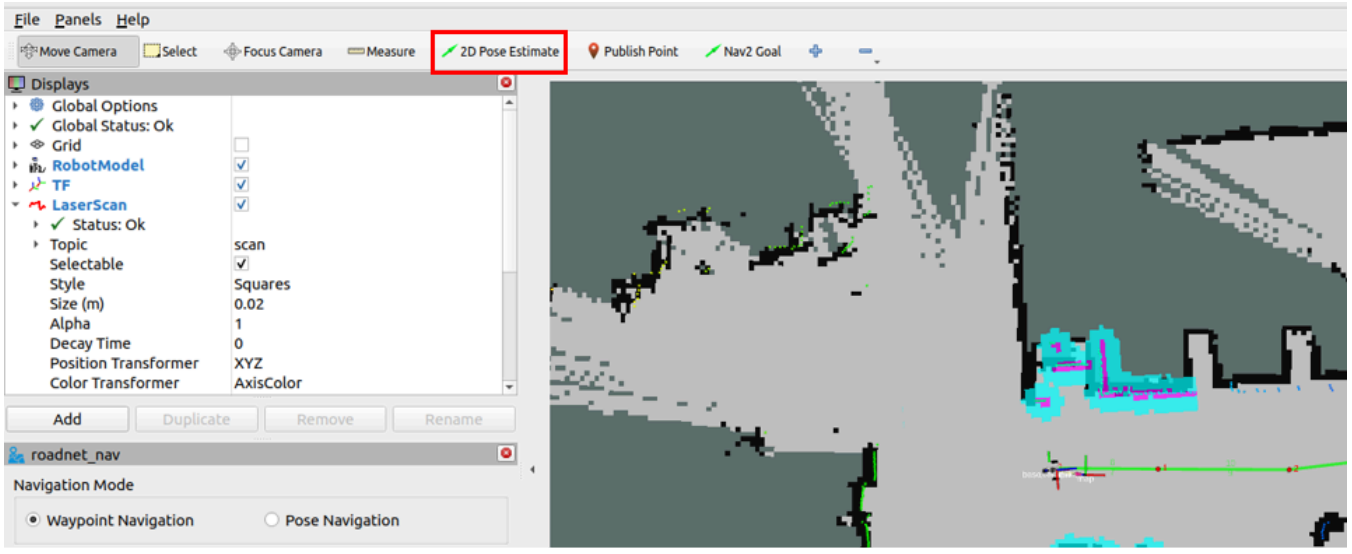
Advanced

- Master the method of requesting navigation through the action service interface
- Master how to debug the navigation parameter file
- Understand the implementation logic of the program source code

2. Preparation

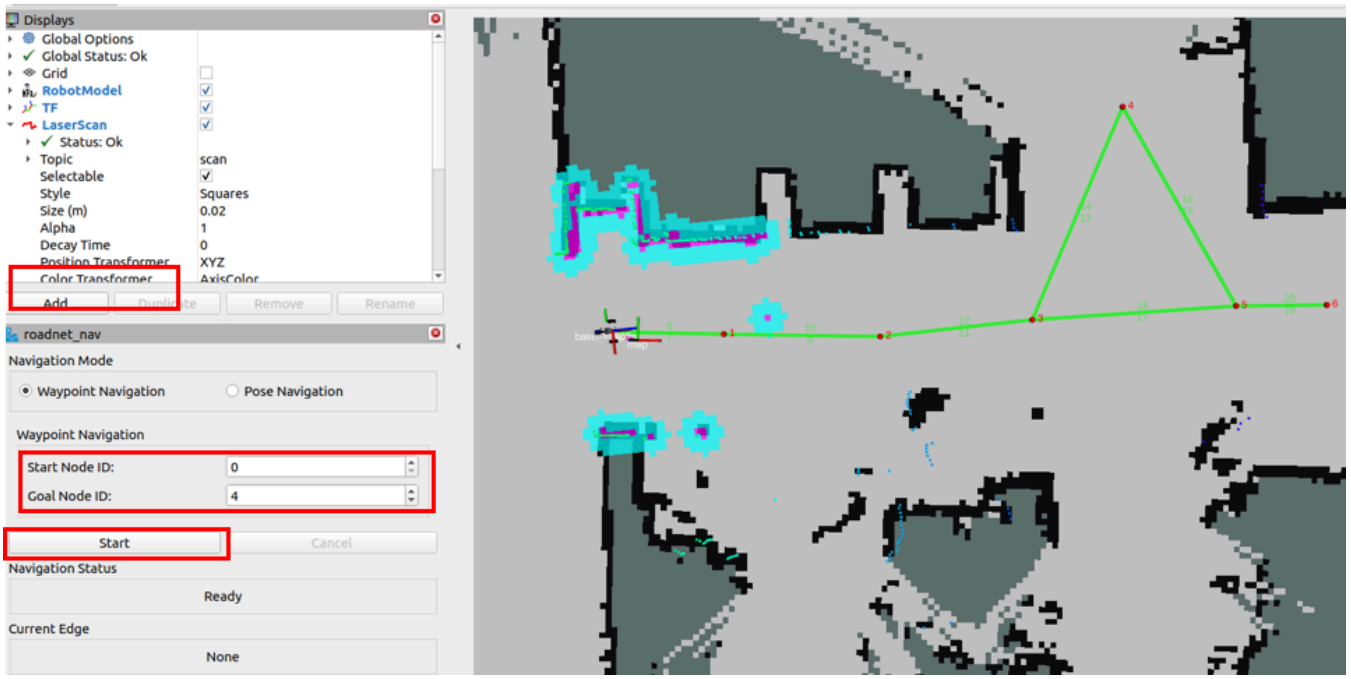
```
ros2 launch yahboomcar_bringup roadnet_nav.launch.py
```

- Click **2D Pose Estimate** to initialize the robot's pose on the map

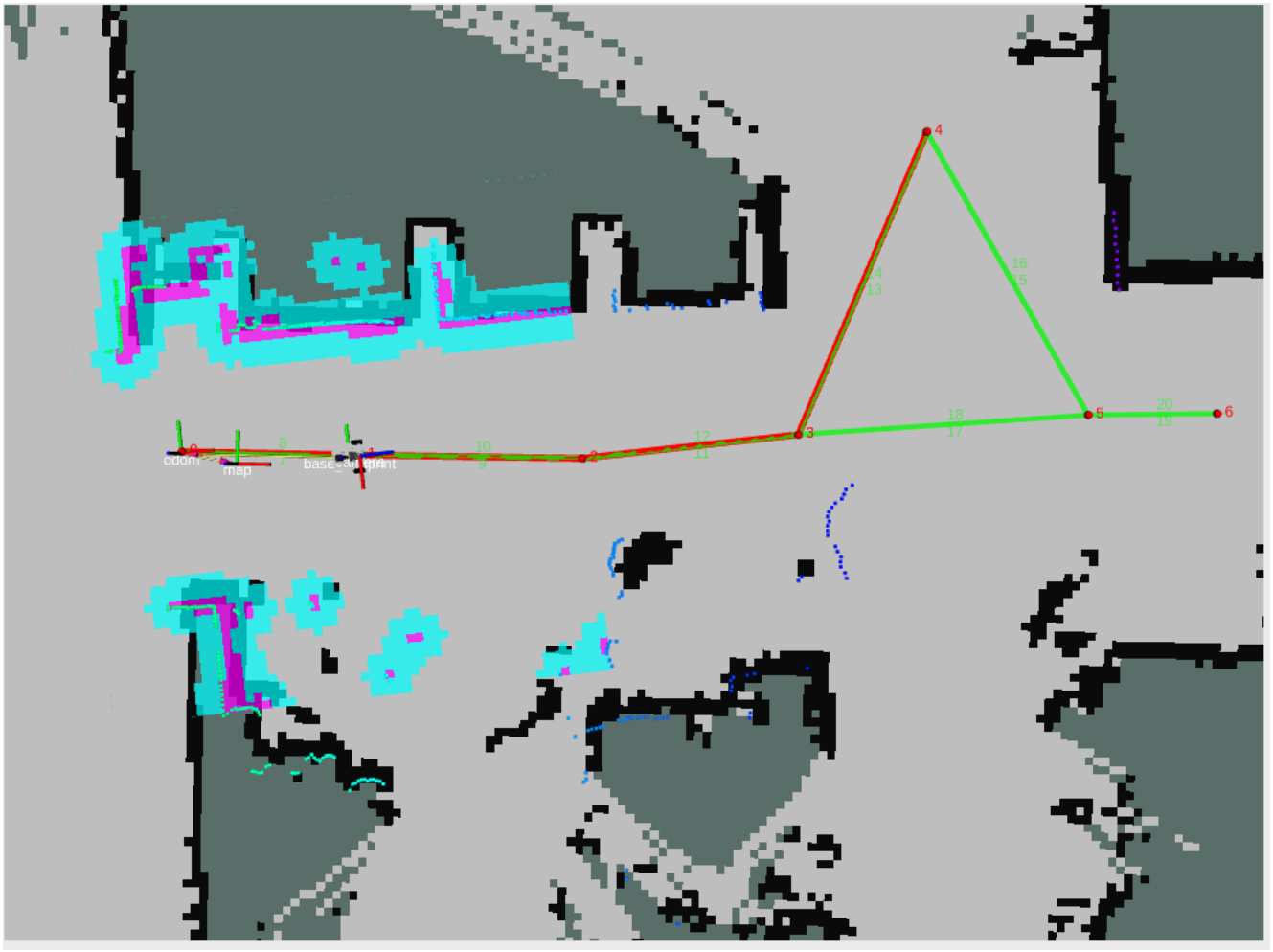


3. Waypoint-to-Waypoint Navigation

- Here we use road network navigation through the roadnet_nav plugin interface. For example, in the figure the robot is close to waypoint 0 and we need to navigate to waypoint 4. Enter the **Start Node ID** and **Goal Node ID**, then click **Start**

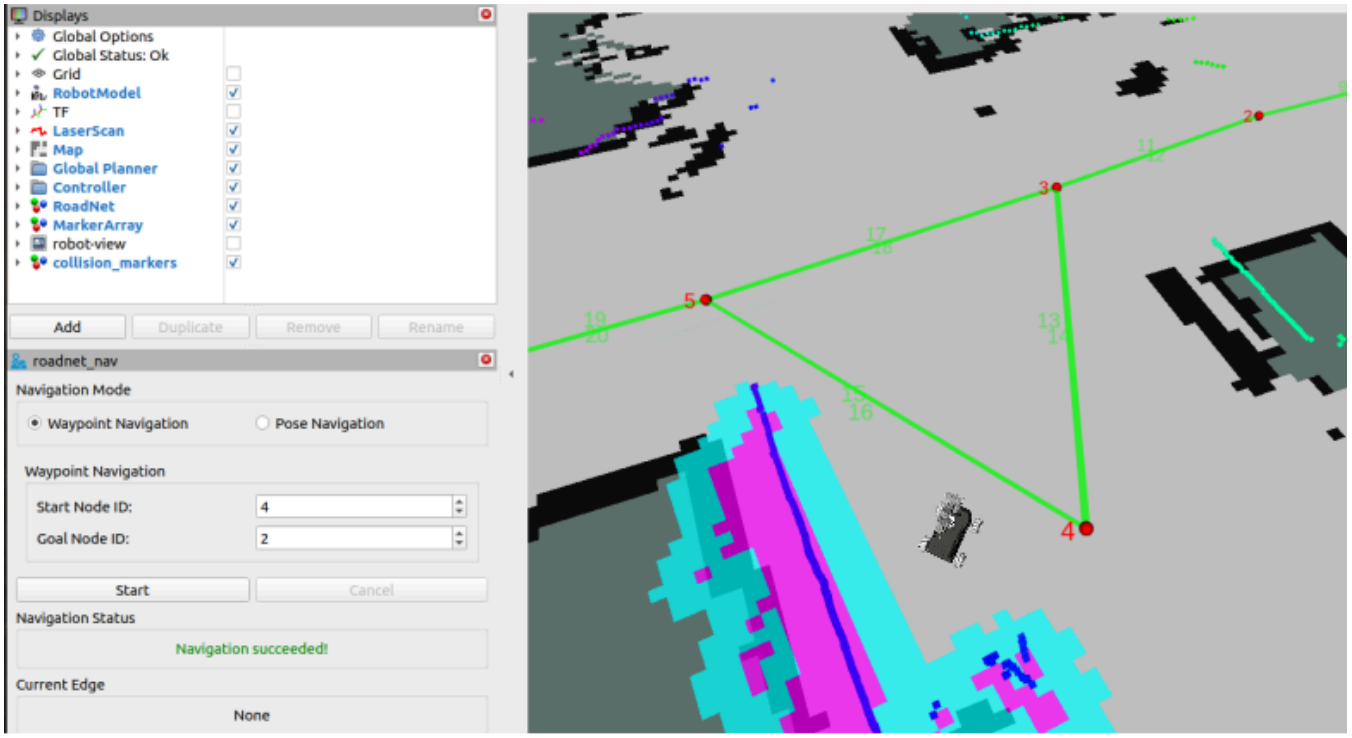


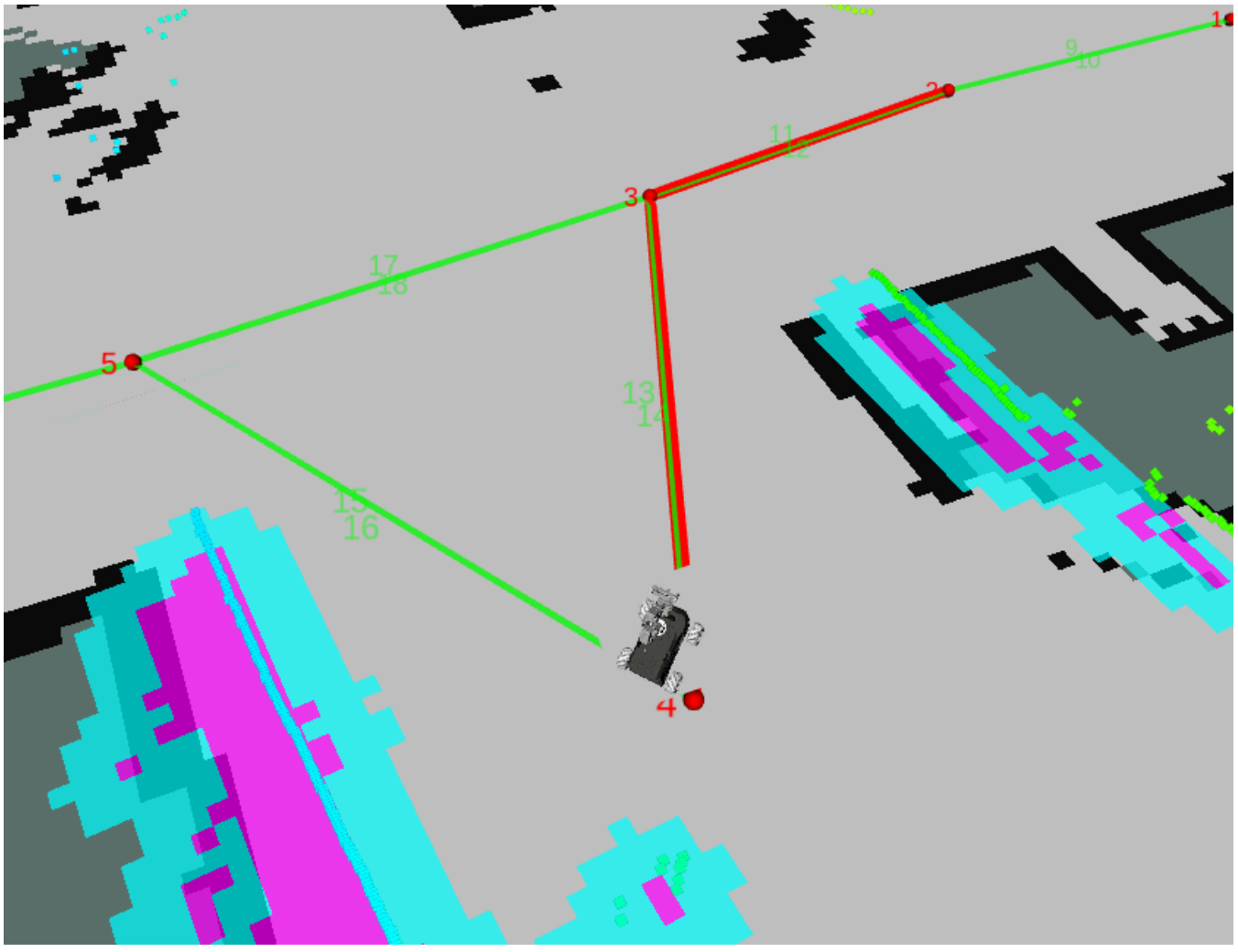
- In RViz you can observe that the global path planned by the robot is generated only within the road network, and the robot follows along this path



3.1 Navigation When the Starting Position Is Not on the Road Network

- If the starting position is not on a point or edge of the road network, then when road network navigation is executed, the robot will automatically first enter the nearest road network edge and then move along the road network. For example, in the case below, the robot is not in the road network; we observe that the nearest waypoint is 4, and here we plan to go to waypoint 2. Enter the start and goal IDs.


Displays
Global Options
Global Status: Ok
Grid
RobotModel
TF
LaserScan
Map
Global Planner
Controller
RoadNet
MarkerArray
robot-view
collision_markers
Add
Duplicate
Remove
Rename
roadnet_nav
Navigation Mode
Waypoint Navigation
Pose Navigation
Waypoint Navigation
Start Node ID:
4
Goal Node ID:
2
Start
Cancel
Navigation Status
Navigation succeeded!
Current Edge
None



3.2 Requesting Road Network Navigation Through the Action Service Interface

> **Tip**
>
> The service interface can be integrated into other programs for feature development

```
ros2 action send_goal /compute_route_and_follow roadnet/action/RoadNetToGoal "{start_id: 4, goal_id: 2}"
```

```
yahboom@yahboom:~$ ros2 action send_goal /compute_route_and_follow roadnet/action/RoadNetToGoal "{start_id: 4, goal_id: 2}"
Waiting for an action server to become available...
Sending goal:
     start_id: 4
start:
  header:
    stamp:
      sec: 0
      nanosec: 0
    frame_id: ''
  pose:
    position:
      x: 0.0
      y: 0.0
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.0
      w: 1.0
goal_id: 2
```

4. Source Code Analysis

- Source code path of the implementation program:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/roadnet/src/route_server.cpp
```

> **Note**
>
> - The roadnet road network feature has a fairly large amount of code, so here the core **computeRouteAndFollow** method is selected for explanation

- Obtain the Action goal request `goal`, initialize the result and feedback objects, and record the start time
- **Outer loop** (replanning loop):
  - Validate the legitimacy of the request, check whether a new goal has preempted it, and clear residual costs in the global costmap
  - Call `findRoute` to compute a feasible path in the road network, and call `densify` to densify the path
  - Wait for the `FollowPath` Action server to be ready, send the dense path, and begin tracking
- **Inner loop** (20 Hz path tracking):
  - Check for cancel/preempt requests, and check whether `FollowPath` has completed
  - Obtain the robot's current pose via TF, and compute the distance to the next waypoint
  - When the distance < `radius_threshold`, determine that the waypoint has been reached, switch to the next edge, and publish progress feedback

    - Call `OperationsManager` to perform obstacle detection and replanning judgment
    - When an anomaly such as a collision is detected, cancel the current `FollowPath`, set the `need_reroute` flag, and break out of the inner loop
- **Replanning handling**: When the outer loop detects `need_reroute`, it uses the start of the blocked edge as the new starting point to replan the path
- **Precise pose navigation**: After road network tracking is complete, if `goal->use_poses` is true, call `NavigateToPose` to navigate to the target precise pose
- **Exception handling**: Catch exceptions such as no valid route, timeout, TF error, no valid graph, and indeterminate nodes, set the corresponding error code, and terminate the Action

```
  void RouteServer::computeRouteAndFollow()
  {
    RCLCPP_INFO(get_logger(), "Computing route and following path to goal.");

    auto goal = compute_route_and_follow_server_->get_current_goal();
    auto result = std::make_shared<RoadNetToGoalResult>();
    auto feedback = std::make_shared<RoadNetToGoalFeedback>();
    ReroutingState rerouting_info;
    auto start_time = this->now();

    try
    {
      Route route;
      nav_msgs::msg::Path path;

      // Reroute trigger conditions: (obstacle detection) or a request to the rerouting
service (ReroutingService)
      while (rclcpp::ok())
      {
        if (!isRequestValid(compute_route_and_follow_server_))
        {
          return;
        }
        // Check whether a new goal has preempted the current one
        if (compute_route_and_follow_server_->is_preempt_requested())
        {
          RCLCPP_INFO(get_logger(), "A new request is received, and the target of the
original request is preempted");
          goal = compute_route_and_follow_server_->accept_pending_goal();
          rerouting_info.reset();
        }
        // Clear residual costs in global_costmap before planning to avoid affecting
planning
        clearGlobalCostmap();

        // Compute a feasible route in the road network
        route = findRoute(goal, rerouting_info);
        RCLCPP_INFO(get_logger(), "Route found with %zu nodes and %zu edges",
                    route.edges.size() + 1u, route.edges.size());
        path = path_converter_->densify(route, rerouting_info, route_frame_, this->now());
```

```
        // Print the path
        std::string route_str;
        if (route.start_node)
        {
          route_str = std::to_string(route.start_node->nodeid);
        }
        for (const auto &edge : route.edges)
        {
          route_str += " ---------->> " + std::to_string(edge->end->nodeid);
        }
        RCLCPP_INFO(get_logger(), "Find route plan: %s", route_str.c_str());

        // Wait for FollowPath action server
        if (!follow_path_client_->wait_for_action_server(std::chrono::seconds(5)))
        {
          RCLCPP_ERROR(get_logger(), "FollowPath action server not available!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath action server not available";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }

        // Send the path to FollowPath
        auto follow_goal = FollowPath::Goal();
        follow_goal.path = path;

        auto send_goal_options = rclcpp_action::Client<FollowPath>::SendGoalOptions();
        auto goal_handle_future = follow_path_client_->async_send_goal(follow_goal,
send_goal_options);

        // Wait for goal acceptance
        if (goal_handle_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal send timed out!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath goal send timed out";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }

        auto follow_goal_handle = goal_handle_future.get();
        if (!follow_goal_handle)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal was rejected!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath goal rejected by controller";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }
        RCLCPP_DEBUG(get_logger(), "FollowPath goal accepted, robot is moving!");

        // Get result future for polling
```

```
        auto follow_result_future = follow_path_client_-
>async_get_result(follow_goal_handle);

        // Track path progress and detect obstacles
        double radius_threshold = 2.0;
        nav2_util::declare_parameter_if_not_declared(
            shared_from_this(), "radius_to_achieve_node",
            rclcpp::ParameterValue(radius_threshold));
        radius_threshold = get_parameter("radius_to_achieve_node").as_double();

        EdgePtr current_edge = nullptr;
        NodePtr next_node = nullptr;
        int edge_idx = -1;
        RouteTrackingState tracking_state;
        tracking_state.last_node = route.start_node;
        tracking_state.next_node = route.start_node;

        // Advance to first edge
        if (!route.edges.empty())
        {
          edge_idx = 0;
          current_edge = route.edges[0];
          next_node = current_edge->end;
          tracking_state.current_edge = current_edge;
          RCLCPP_INFO(get_logger(),
                      "Entering edge:%u  id:%u ---------------------> id:%u",
                      current_edge->edgeid,
                      current_edge->start->nodeid,
                      current_edge->end->nodeid);
        }
        rclcpp::Rate rate(20.0);
        bool follow_completed = false;
        bool route_tracking_done = route.edges.empty();
        bool need_reroute = false;

        while (rclcpp::ok() && !follow_completed && !need_reroute)
        {
          // Check if our action was cancelled
          if (compute_route_and_follow_server_->is_cancel_requested())
          {
            RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            compute_route_and_follow_server_->terminate_all();
            return;
          }

          // Check if preempted by a new goal
          if (compute_route_and_follow_server_->is_preempt_requested())
          {
            RCLCPP_INFO(get_logger(), "Goal preempted, cancelling FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            break;  // Break out of the inner loop; the outer loop will handle the new goal
          }
```

```
          // Check if FollowPath completed
          if (follow_result_future.wait_for(std::chrono::milliseconds(0)) ==
              std::future_status::ready)
          {
            auto wrapped_result = follow_result_future.get();
            if (wrapped_result.code == rclcpp_action::ResultCode::SUCCEEDED)
            {
              RCLCPP_INFO(get_logger(), "FollowPath completed successfully!");
            }
            else if (wrapped_result.code == rclcpp_action::ResultCode::CANCELED)
            {
              RCLCPP_WARN(get_logger(), "FollowPath was cancelled.");
            }
            else
            {
              RCLCPP_WARN(get_logger(), "FollowPath failed (code=%d).",static_cast<int>
(wrapped_result.code));
            }
            follow_completed = true;
            break;
          }

          // Get the robot's current position
          geometry_msgs::msg::PoseStamped robot_pose;
          bool got_pose = nav2_util::getCurrentPose(
              robot_pose, *tf_, route_frame_, base_frame_, 0.5);

          // Track edge transitions
          if (!route_tracking_done && next_node && got_pose)
          {
            const double dx = next_node->coords.x - robot_pose.pose.position.x;
            const double dy = next_node->coords.y - robot_pose.pose.position.y;
            const double dist = std::sqrt(dx * dx + dy * dy);

            if (dist < radius_threshold)
            {
              // Node achieved
              RCLCPP_INFO(get_logger(),
                          "Arrived at node id:%u (%.2f, %.2f)",
                          next_node->nodeid,
                          next_node->coords.x, next_node->coords.y);

              tracking_state.last_node = next_node;
              edge_idx++;

              if (edge_idx < static_cast<int>(route.edges.size()))
              {
                current_edge = route.edges[edge_idx];
                next_node = current_edge->end;
                tracking_state.current_edge = current_edge;
                tracking_state.next_node = next_node;
                tracking_state.route_edges_idx = edge_idx;
```

```
                RCLCPP_INFO(get_logger(),
                            "Entering edge:%u  id:%u ---------------------> id:%u",
                            current_edge->edgeid,
                            current_edge->start->nodeid,
                            current_edge->end->nodeid);

                // Publish feedback on edge transition
                feedback->last_node_id = tracking_state.last_node ?
tracking_state.last_node->nodeid : 0;
                feedback->next_node_id = next_node ? next_node->nodeid : 0;
                feedback->current_edge_id = current_edge ? current_edge->edgeid : 0;
                feedback->rerouted = false;
                compute_route_and_follow_server_->publish_feedback(feedback);
              }
              else
              {
                // All nodes achieved
                route_tracking_done = true;
                tracking_state.current_edge = nullptr;
                tracking_state.next_node = nullptr;
                RCLCPP_INFO(get_logger(), "All route nodes achieved!");
              }
            }
          }

          // Run OperationsManager to perform obstacle detection and replanning judgment
          if (got_pose && operations_manager_)
          {
            bool status_change = (edge_idx >= 0 && tracking_state.last_node != nullptr);
            OperationsResult ops_result = operations_manager_->process(
                status_change, tracking_state, route, robot_pose, rerouting_info);

            // If the operation triggered a replanning request
            if (ops_result.reroute)
            {
              RCLCPP_WARN(get_logger(), "Rerouting requested by operations (e.g., collision
detected)!");

              // Save the replanning information
              if (!ops_result.blocked_ids.empty())
              {
                if (!aggregate_blocked_ids_) {
                  rerouting_info.blocked_ids = ops_result.blocked_ids;
                } else {
                  rerouting_info.blocked_ids.insert(
                      rerouting_info.blocked_ids.end(),
                      ops_result.blocked_ids.begin(), ops_result.blocked_ids.end());
                }
              }

              // Set the replanning start point to the start node of the current blocked
edge, so the robot rolls back to a safe node and then replans
```

```
              NodePtr reroute_start_node = nullptr;
              if (tracking_state.last_node)
              {
                reroute_start_node = tracking_state.last_node;
              }
              else if (tracking_state.current_edge)
              {
                reroute_start_node = tracking_state.current_edge->start;
              }

              if (reroute_start_node)
              {
                rerouting_info.rerouting_start_id = reroute_start_node->nodeid;
                rerouting_info.rerouting_start_pose.pose.position.x = reroute_start_node-
>coords.x;
                rerouting_info.rerouting_start_pose.pose.position.y = reroute_start_node-
>coords.y;
                rerouting_info.rerouting_start_pose.header.frame_id = route_frame_;
                rerouting_info.rerouting_start_pose.header.stamp = this->now();
              }
              else
              {
                rerouting_info.rerouting_start_id = std::numeric_limits<unsigned
int>::max();
                rerouting_info.rerouting_start_pose = geometry_msgs::msg::PoseStamped();
              }

              // Update so during rerouting we can check if we are continuing on the same
edge
              rerouting_info.curr_edge = tracking_state.current_edge;

              // Cancel the current FollowPath
              follow_path_client_->async_cancel_goal(follow_goal_handle);
              need_reroute = true;

              // Publish feedback with reroute info
              feedback->last_node_id = tracking_state.last_node ? tracking_state.last_node-
>nodeid : 0;
              feedback->next_node_id = next_node ? next_node->nodeid : 0;
              feedback->current_edge_id = current_edge ? current_edge->edgeid : 0;
              feedback->operations_triggered = ops_result.operations_triggered;
              feedback->rerouted = true;
              compute_route_and_follow_server_->publish_feedback(feedback);
              break;
            }
          }

          rate.sleep();
        }

        // If replanning is needed, continue the outer loop
        if (need_reroute)
        {
```

```
          RCLCPP_INFO(get_logger(), "Rerouting from node %u (%.2f, %.2f)...",
              rerouting_info.rerouting_start_id,
              rerouting_info.rerouting_start_pose.pose.position.x,
              rerouting_info.rerouting_start_pose.pose.position.y);
          continue;
        }

        // If FollowPath completed or was cancelled, break out of the loop
        if (follow_completed || compute_route_and_follow_server_->is_cancel_requested())
        {
          break;
        }

        // If preempted, continue the outer loop to handle the new goal
        if (compute_route_and_follow_server_->is_preempt_requested())
        {
          continue;
        }
      }

      // Check whether it was cancelled
      if (compute_route_and_follow_server_->is_cancel_requested())
      {
        compute_route_and_follow_server_->terminate_all();
        return;
      }

      // If using pose navigation, navigate to goal pose after route following
      if (goal->use_poses)
      {
        RCLCPP_INFO(get_logger(), "Navigating to goal pose...");

        // Wait for NavigateToPose action server
        if (!nav_to_pose_client_->wait_for_action_server(std::chrono::seconds(5)))
        {
          RCLCPP_WARN(get_logger(), "NavigateToPose action server not available, skipping
precise navigation.");
        }
        else
        {
          // Create NavigateToPose goal
          auto nav_goal = NavigateToPose::Goal();
          nav_goal.pose = goal->goal;

          // Send goal to NavigateToPose
          auto nav_goal_options = rclcpp_action::Client<NavigateToPose>::SendGoalOptions();
          auto nav_goal_future = nav_to_pose_client_->async_send_goal(nav_goal,
nav_goal_options);

          // Wait for goal acceptance
          if (nav_goal_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
          {
```

```
            RCLCPP_WARN(get_logger(), "NavigateToPose goal send timed out, skipping precise
navigation.");
          }
          else
          {
            auto nav_goal_handle = nav_goal_future.get();
            if (!nav_goal_handle)
            {
              RCLCPP_WARN(get_logger(), "NavigateToPose goal was rejected, skipping precise
navigation.");
            }
            else
            {
              RCLCPP_INFO(get_logger(), "NavigateToPose goal accepted, robot is moving to
precise goal pose!");

              // Get result future for polling
              auto nav_result_future = nav_to_pose_client_-
>async_get_result(nav_goal_handle);

              // Wait for NavigateToPose to complete
              rclcpp::Rate nav_rate(20.0);
              bool nav_completed = false;

              while (rclcpp::ok() && !nav_completed)
              {
                // Check if our action was cancelled
                if (compute_route_and_follow_server_->is_cancel_requested())
                {
                  RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling NavigateToPose...");
                  nav_to_pose_client_->async_cancel_goal(nav_goal_handle);
                  compute_route_and_follow_server_->terminate_all();
                  return;
                }

                // Check if preempted by a new goal
                if (compute_route_and_follow_server_->is_preempt_requested())
                {
                  RCLCPP_INFO(get_logger(), "Goal preempted, cancelling NavigateToPose...");
                  nav_to_pose_client_->async_cancel_goal(nav_goal_handle);
                  // Recursively handle new goal
                  computeRouteAndFollow();
                  return;
                }

                // Check if NavigateToPose completed
                if (nav_result_future.wait_for(std::chrono::milliseconds(0)) ==
                    std::future_status::ready)
                {
                  auto nav_result = nav_result_future.get();
                  if (nav_result.code == rclcpp_action::ResultCode::SUCCEEDED)
                  {
                    RCLCPP_INFO(get_logger(), "NavigateToPose completed successfully!");
```

```
                }
                else if (nav_result.code == rclcpp_action::ResultCode::CANCELED)
                {
                  RCLCPP_WARN(get_logger(), "NavigateToPose was cancelled.");
                }
                else
                {
                  RCLCPP_WARN(get_logger(), "NavigateToPose failed (code=%d).",
                              static_cast<int>(nav_result.code));
                }
                nav_completed = true;
                break;
              }

              nav_rate.sleep();
            }
          }
        }
      }

      // Return result
      result->success = true;
      result->execution_duration = this->now() - start_time;
      RCLCPP_INFO(get_logger(), "========== Route and follow completed! ==========");
      compute_route_and_follow_server_->succeeded_current(result);
    }
    catch (nav2_core::NoValidRouteCouldBeFound &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::NO_VALID_ROUTE;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::TimedOut &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::TIMEOUT;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::RouteTFError &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::TF_ERROR;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::NoValidGraph &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::NO_VALID_GRAPH;
      result->error_msg = ex.what();
```

```
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::IndeterminantNodesOnGraph &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::INDETERMINANT_NODES_ON_GRAPH;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (std::exception &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::UNKNOWN;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
```

5. Parameter Debugging

- Road network navigation parameter file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/roadnet_nav.yaml
```

5.1 Navigation Controller Debugging

> **Tip**
>
> **Parameter Adjustment Guide**
>
> - If you need to adjust the movement speed and rotational angular velocity during navigation, adjust `desired_linear_vel` and `max_angular_accel`
> - If you need to adjust the precision of reaching the goal point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
> - For adjustments to other parameters, see the comments below

```
controller_server:
  ros__parameters:
    # ==================== Controller Core Global Parameters ====================
    # The controller's main loop frequency (Hz), which determines the publishing frequency
of local path planning and velocity commands; it must match the hardware execution frequency
    controller_frequency: 20.0
    # Costmap update timeout (seconds). If the costmap is not updated within this time, the
controller stops outputting velocity commands to avoid planning based on an outdated map
    costmap_update_timeout: 0.30
    # Minimum velocity threshold in the X direction (forward/backward) (m/s). Velocity
commands below this value are considered invalid, used to filter out tiny jitter
    min_x_velocity_threshold: 0.001
```

```
    # Minimum velocity threshold in the Y direction (lateral movement) (m/s). Only applies
to omnidirectional robots (such as Mecanum wheels); differential-drive robots can set a
larger value (e.g. 0.5) to disable lateral movement
    min_y_velocity_threshold: 0.5
    # Minimum velocity threshold for rotation (rad/s). Rotation commands below this value
are considered invalid, filtering out tiny rotational jitter
    min_theta_velocity_threshold: 0.001
    # Controller failure tolerance (meters). When the deviation of the locally planned path
exceeds this value, the controller execution is judged to have failed and replanning is
triggered
    failure_tolerance: 0.3
    # List of progress checker plugins, used to monitor whether the robot is moving toward
the goal
    progress_checker_plugins: ["progress_checker"]
    # List of goal checker plugins, used to determine whether the robot has reached the goal
pose
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugins; the core is the local path tracking algorithm. Here the
pure pursuit controller corresponding to FollowPath is used
    controller_plugins: ["FollowPath"]
    # Whether to use real-time priority scheduling for the controller thread
    use_realtime_priority: false
    # Name of the speed limit topic. The controller subscribes to velocity commands on this
topic to dynamically adjust the maximum linear/angular velocity (e.g., slow down when
avoiding obstacles)
    speed_limit_topic: "speed_limit"

    # ====================== Progress Checker Parameters (SimpleProgressChecker)
====================
    progress_checker:
      # Progress checker plugin type. SimpleProgressChecker is the basic implementation,
monitoring the robot's movement distance and time
      plugin: "nav2_controller::SimpleProgressChecker"
      # Minimum movement distance threshold for the robot (meters). If it does not move
beyond this distance within movement_time_allowance, it is judged to have made no progress
      required_movement_radius: 0.05
      # Maximum allowed time without progress (seconds). If required_movement_radius is
still not met after this time, the controller aborts the task (error code 105)
      movement_time_allowance: 10.0

    # ====================== Goal Checker Parameters (SimpleGoalChecker)
====================
    general_goal_checker:
      # Whether it is a stateful checker. When set to True, it continuously monitors the
goal pose until the tolerance is met; when False, it only checks once
      stateful: True
      # Goal checker plugin type. SimpleGoalChecker is the basic implementation, determining
whether the position and angle are within the tolerance range
      plugin: "nav2_controller::SimpleGoalChecker"
      # XY-plane position tolerance (meters). When the deviation between the robot's
position and the goal position is smaller than this value, the position is judged to be
reached
      xy_goal_tolerance: 0.08
```

```yaml
      # Heading angle tolerance (radians). When the deviation between the robot's heading and the goal heading is smaller than this value, the angle is judged to be reached
      yaw_goal_tolerance: 0.03

    # ==================== Pure Pursuit Controller Parameters (RegulatedPurePursuitController) ====================
    FollowPath:
      # Local path tracking plugin type. RegulatedPurePursuitController is a pure pursuit algorithm with velocity regulation, suitable for differential-drive robots
      plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
      # Desired linear velocity (m/s). The target linear velocity during normal robot travel; it is dynamically scaled by obstacle avoidance and curvature regulation
      desired_linear_vel: 0.2
      # Lookahead distance (meters). The core parameter of the pure pursuit algorithm, determining the distance of the path point the robot tracks; it must match the robot's velocity
      lookahead_dist: 0.5
      # Minimum lookahead distance (meters). Prevents the robot from turning too frequently when the lookahead distance is too small
      min_lookahead_dist: 0.05
      # Maximum lookahead distance (meters). Prevents the robot from reacting sluggishly when the lookahead distance is too large
      max_lookahead_dist: 0.9
      # Lookahead time (seconds). If use_velocity_scaled_lookahead_dist is enabled, lookahead distance = desired_linear_vel * lookahead_time
      lookahead_time: 1.5
      # Angular velocity for rotating to the goal heading (rad/s). The rotation speed used to adjust the heading after the robot reaches the position
      rotate_to_heading_angular_vel: 1.3
      # TF transform tolerance (seconds). The maximum allowed deviation between the timestamp of the robot pose transform and the current time, to handle TF delay
      transform_tolerance: 0.5
      # Whether to use a velocity-scaled lookahead distance. When set to True, the lookahead distance changes dynamically with the robot's velocity (the faster the speed, the farther the lookahead)
      use_velocity_scaled_lookahead_dist: false
      # Minimum linear velocity when approaching the goal (m/s). When the robot is close to the goal, it slows down to this value to avoid overshooting the goal
      min_approach_linear_velocity: 0.05
      # Approach velocity scaling distance (meters). When the robot is closer to the goal than this value, it begins to decelerate linearly (from desired_linear_vel to min_approach_linear_velocity)
      approach_velocity_scaling_dist: 1.0
      # Whether to enable collision detection. When set to True, the controller checks for collisions on the lookahead path to avoid the robot hitting obstacles
      use_collision_detection: false
      # Maximum allowed time to collision up to the lookahead point (seconds). Takes effect only when use_collision_detection is True, used to judge collision risk
      max_allowed_time_to_collision_up_to_carrot: 1.0
      # Whether to enable linear velocity regulation based on path curvature. When set to True, the smaller the turning radius, the lower the linear velocity
      use_regulated_linear_velocity_scaling: true
```

```yaml
      # Whether to enable linear velocity regulation based on the costmap. When set to True, the closer the obstacle, the lower the linear velocity
      use_cost_regulated_linear_velocity_scaling: false
      # Minimum turning radius that triggers curvature regulation (meters). When the turning radius corresponding to the path curvature is smaller than this value, deceleration begins
      regulated_linear_scaling_min_radius: 0.05
      # Minimum velocity for curvature regulation (m/s). Even if the turning radius is extremely small, the linear velocity will not drop below this value
      regulated_linear_scaling_min_speed: 0.25
      # Whether to use a fixed-curvature lookahead distance. When set to True, the lookahead distance is determined by curvature_lookahead_dist
      use_fixed_curvature_lookahead: false
      # Curvature lookahead distance (meters). Takes effect only when use_fixed_curvature_lookahead is True
      curvature_lookahead_dist: 1.0
      # Whether to enable "rotate to heading first, then move". When set to True, if the robot's heading deviates too much from the path direction, it rotates first and then moves forward
      use_rotate_to_heading: true
      # Minimum angle that triggers rotation to heading (radians). When the deviation between the robot's heading and the path direction exceeds this value, rotation adjustment begins
      rotate_to_heading_min_angle: 0.785 # about 45 degrees
      # Maximum angular acceleration (rad/s²). Limits the robot's rotation acceleration to avoid hardware damage caused by sharp turns
      max_angular_accel: 1.2
      # Maximum distance for robot pose search (meters). When the controller cannot find a matching robot pose on the path, it searches for path points within this distance
      max_robot_pose_search_dist: 10.0
      # Whether to interpolate curvature after the goal point. When set to True, the path after the goal point is interpolated with curvature to avoid the robot stopping abruptly upon reaching the goal
      interpolate_curvature_after_goal: false
      # Cost scaling distance (meters). Takes effect only when use_cost_regulated_linear_velocity_scaling is True, used to compute the weight for obstacle-avoidance deceleration
      cost_scaling_dist: 0.08
      # Cost scaling gain. Takes effect only when use_cost_regulated_linear_velocity_scaling is True; the larger the gain, the more pronounced the obstacle-avoidance deceleration
      cost_scaling_gain: 1.0
      # Inflation cost scaling factor, used to convert the inflation layer cost of the costmap into a velocity scaling coefficient; the larger the value, the more sensitive the obstacle avoidance of the inflation zone
      inflation_cost_scaling_factor: 3.0
```

5.2 Obstacle Avoidance and Costmap Debugging

> **Tip**
>
> - If the robot gets too close to obstacles in the map during navigation, you can make the robot stay farther from obstacles by adjusting the inflation area — increase `inflation_radius` (inflation radius) and `cost_scaling_factor` (scaling factor; the larger the value, the less likely the robot is to get close to obstacles) in local_costmap and global_costmap
> - For adjustments to other parameters, see the comments below

```
# Local costmap configuration: a short-term costmap used for real-time obstacle avoidance by the robot, which scrolls and updates as the robot moves
local_costmap:
  local_costmap:
    ros__parameters:
      # ==================== Costmap Core Global Parameters ====================
      # Costmap update frequency (Hz), determining the refresh rate of the local map; it must match the sensor frequency
      update_frequency: 5.0
      # Costmap publish frequency (Hz), the frequency of publishing the map to visualization tools such as RViz; lower than the update frequency to reduce bandwidth usage
      publish_frequency: 2.0
      # Global coordinate frame of the costmap; the local map uses the odom frame
      global_frame: odom
      # Robot base coordinate frame, consistent with the robot center coordinate frame configured for the hardware
      robot_base_frame: base_link
      # Whether to enable rolling window mode, the core feature of the local map: the map scrolls as the robot moves, keeping only the map area around the robot
      rolling_window: true
      # Width of the rolling window (meters), the lateral extent of the local map
      width: 3
      # Height of the rolling window (meters), the longitudinal extent of the local map
      height: 3
      # Costmap resolution (meters/pixel), representing the actual distance each pixel represents; 0.05 means 5 cm/pixel
      resolution: 0.05
      # Robot footprint (polygon), representing the robot's actual shape as a list of coordinates
      footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
      # Padding distance for the robot footprint (meters), an extra safety distance expanded beyond the actual footprint, used as a buffer during obstacle avoidance
      footprint_padding: 0.0616
      # List of layer plugins for the costmap, loaded in order (voxel_layer: voxel obstacle layer; inflation_layer: inflation layer)
      plugins: ["voxel_layer", "inflation_layer"]
      # List of filter plugins for the costmap, used for post-processing the map (keepout_filter: keepout zone filtering)
      filters: ["keepout_filter"]
```

```yaml
      # Whether to always publish the full costmap. When set to True, the full map is
published on every update; when False, only changed areas are published (reducing bandwidth)
      always_send_full_costmap: True
      # Service introspection mode; disabled means service introspection is disabled;
options are enabled/verbose
      service_introspection_mode: "disabled"

      # ==================== Filter Plugin: Keepout Filter (KeepoutFilter)
====================
      keepout_filter:
        # Filter plugin type; KeepoutFilter is used to mark keepout zones in the costmap
        plugin: "nav2_costmap_2d::KeepoutFilter"
        # Whether to enable keepout zone filtering; KEEPOUT_ZONE_ENABLED is an external
parameter (must be assigned true/false at startup)
        enabled: KEEPOUT_ZONE_ENABLED
        # Subscription topic for keepout zone information, receiving the geometry and
position information of keepout zones (such as PolygonStamped)
        filter_info_topic: "keepout_costmap_filter_info"
        # Whether to override the lethal cost of keepout zones. When set to True, a custom
cost replaces the default lethal cost (255)
        override_lethal_cost: True
        # Custom lethal cost for keepout zones; 200 means lower than the default lethal cost
(255), but still a high-cost keepout zone
        lethal_override_cost: 200

      # ==================== Layer Plugin: Inflation Layer (InflationLayer)
====================
      inflation_layer:
        # Layer plugin type; InflationLayer is used to generate an inflation area around
obstacles to prevent the robot from colliding
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor, controlling the cost decay rate of the inflation area (the
larger the value, the faster the cost decays, and the smaller the inflation area)
        cost_scaling_factor: 3.0
        # Inflation radius (meters), the inflation range around obstacles, i.e. the minimum
safe distance from the robot footprint to obstacles
        inflation_radius: 0.1

# Global costmap configuration: a long-term costmap used for the robot's global path
planning, covering the entire environment
global_costmap:
  global_costmap:
    ros__parameters:
      # ==================== Costmap Core Global Parameters ====================
      # Costmap update frequency (Hz); the global map does not need high-frequency updates,
1-2 Hz is sufficient
      update_frequency: 1.0
      # Costmap publish frequency (Hz); consistent with the update frequency is sufficient
      publish_frequency: 1.0
      # Global coordinate frame of the costmap; the global map uses the map frame (globally
consistent, suitable for path planning)
      global_frame: map
      # Robot base coordinate frame, consistent with the local map
```

```
      robot_base_frame: base_link
      # Robot radius (replaced by footprint)
      # robot_radius: 0.22
      # Robot footprint (consistent with the local map)
      footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
      # Padding distance for the robot footprint (consistent with the local map)
      footprint_padding: 0.0616
      # Costmap resolution (consistent with the local map to ensure coordinate matching)
      resolution: 0.05
      # Whether to track unknown space. When set to True, unexplored areas are marked in the
map (used for SLAM and planning in unknown environments)
      track_unknown_space: true
      # List of layer plugins for the costmap (static_layer: static layer; obstacle_layer:
2D obstacle layer; inflation_layer: inflation layer)
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      # List of filter plugins for the costmap (keepout_filter: keepout zone filtering;
speed_filter: speed limit filtering)
      filters: ["keepout_filter", "speed_filter"]
      # Whether to always publish the full costmap
      always_send_full_costmap: True
      # Service introspection mode
      service_introspection_mode: "disabled"

      inflation_layer:
        # Layer plugin type; InflationLayer is used to generate an inflation area around
obstacles to prevent the robot from colliding
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor, controlling the cost decay rate of the inflation area (the
larger the value, the faster the cost decays, and the smaller the inflation area)
        cost_scaling_factor: 3.0
        # Inflation radius (meters), the inflation range around obstacles, i.e. the minimum
safe distance from the robot footprint to obstacles
        inflation_radius: 0.1
```